

Inteligencia artificial avanzada para la ciencia de datos

Alejandro Adrián García Martínez

Escuela de ingeniería y Ciencias

30 de septiembre del 2026

Abstracto— Este documento se enfoca en la implementación manual de un algoritmo de regresión logística utilizando una gradiente descendiente sin la asistencia de bibliotecas externas. El objetivo es desarrollar un modelo capaz de predecir con precisión si un alumno es un caso de depresión en base a distintos parámetros en su vida diaria. Se realizó un Análisis Exploratorio de Datos (EDA) para entender y limpiar los datos y permitir que el modelo los pueda interpretar de forma correcta

I. INTRODUCCIÓN

La inteligencia artificial ha evolucionado a pasos agigantados en los últimos años, pasando a volverse una herramienta de uso diario por millones de personas alrededor del mundo. Originalmente se utilizaba para crear sistemas que pudieran realizar tareas que requerían de la intervención humana, hoy en día esta se ha diversificado en distintos campos de investigación en diferentes áreas del conocimiento, uno de ellos el conocido como “Machine Learning”.

Este tipo de inteligencia artificial es capaz de aprender de los datos y mejorar su rendimiento sin tener una tarea en concreto dándole flexibilidad para poder ser utilizada en un sinnúmero de proyectos. Esta busca patrones para poder tomar decisiones y se clasifica en 3 categorías: aprendizaje supervisado, no supervisado y por esfuerzo. El modelo presentado en este documento es de tipo “Aprendizaje Supervisado” en el que se entrena un modelo con un conjunto de datos y se sabe la respuesta correcta, el modelo intenta predecir la respuesta y se compara la predicción con la real, finalmente se ajustan los parámetros del modelo para acercarse a acertar con mayor frecuencia.

II. CONTEXTO DEL PROBLEMA

A. Origen

Los datos por utilizar en este documento son recopilados de la página “Kaggle”. Estos datos recopilan las respuestas de distintos jóvenes de diferentes ciudades y contestan preguntas relacionadas a sus hábitos y esfuerzo en los estudios o trabajo.

Algunos de estos jóvenes sufren de depresión un trastorno mental que afecta la salud emocional y mental del individuo que lo sufre, algunos de los síntomas conocidos son sentimientos de tristeza, fatiga, cambios en el sueño, alteraciones en el apetito, entre otros. De no ser tratado de forma correcta puede llevar al individuo a la muerte.

El objetivo de estos datos es poder entrenar un modelo de “Machine Learning” que al conocer la respuesta a ciertos hábitos de un individuo pueda predecir si la persona sufre de depresión.

B. Base de datos

Los datos consisten en 27,901 instancias repartidas en 18 columnas o “features”. Estas columnas representan la información de la siguiente forma:

Id	Identificador único	Identificador único
Gender	Genero del individuo	Male Female
Age	Edad del individuo	Int64
City	Ciudad donde reside	Str
Profession	A que se dedican	Str
Academic Pressure	Cuanta presión sienten por parte de la escuela	0 a 5 0: Poco o nada 5: Mucho
Work Pressure	Presión por parte del trabajo	0 a 5 0: Poco o nada 5: Mucho
CGPA	Promedio académico	Float64
Study Satisfaction	Satisfacción con sus estudios	0 a 5 0: Poco o nada 5: Mucho
Job Satisfaction	Satisfacción con su trabajo	0 a 5 0: Poco o nada 5: Mucho
Sleep Duration	Duración del sueño	Less than 5 hours 5-6 hours 7-8 hours More than 8 hours
Dietary Habits	Hábitos alimenticios	Unhealthy Moderate Healthy

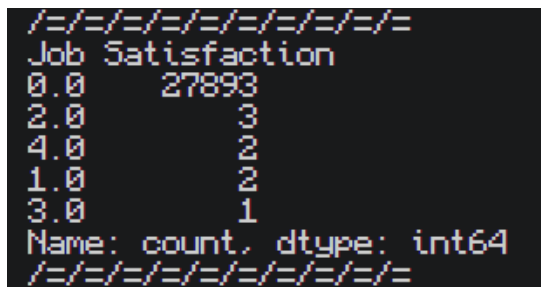


Fig. 4 Conteo de valores en la columna "Job Satisfaction"

La columna de "Work/Study Hours" generaliza ambos tipos de respuesta por lo que se puede conservar, si estuvieran separados se borraría la relacionada al trabajo.

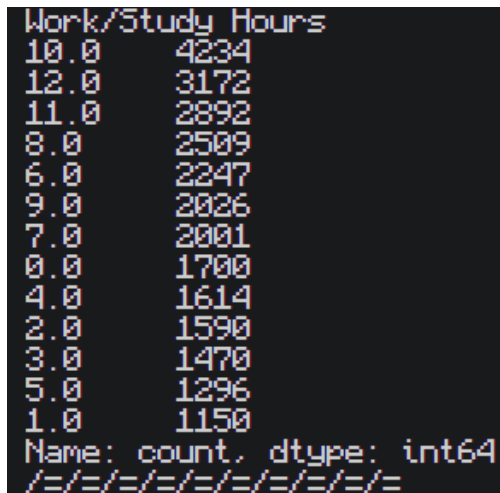


Fig. 5 Conteo de valores en la columna "Work/Study Hours"

3) Datos tipo "str"

Para las columnas de "City" y "Degree" se hizo profundizo un poco más sobre que representaban estos datos y si convenía mantenerlos y de que forma. Se realizó la siguiente grafica para poder tener una representación visual de cómo se comportan los datos.

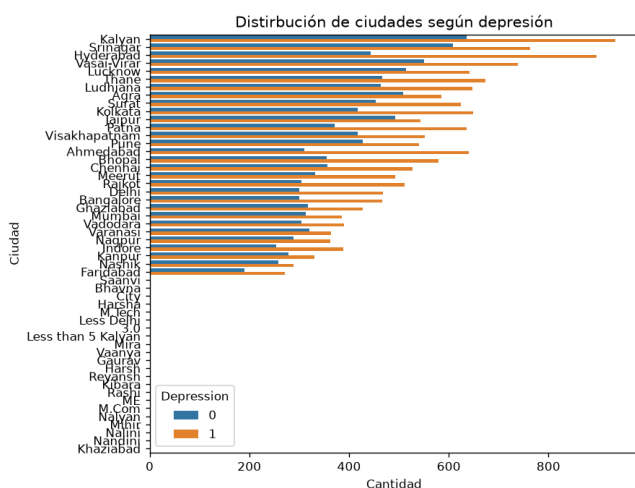


Fig. 6 Distribución de ciudades según depresión

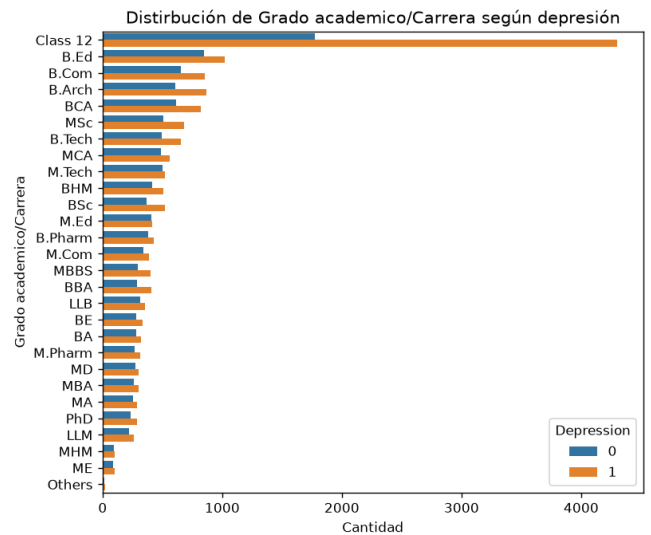


Fig. 7 Distribución de carreras según depresión

Como se puede ver en la gráfica de las carreras la gran cantidad de los datos se encuentran en el valor "Class 12" esto nos indica no realmente que si se encuentra en la clase 12 tiende a tener más probabilidades de presentar depresión, si no, como fueron recabados los datos. Recordemos que la gran mayoría son estudiantes por lo que pocos han de tener un rango académico mayor a estudiante, como se puede ver en la gráfica. Por lo que estos datos nos indican como fue tomada la muestra no un indicador real.

Algo similar sucede con las ciudades, esos datos representan las regiones geográficas donde se recopilaron los datos no indican que al provenir o residir en cierta ciudad es más probable de padecer depresión. Es por eso que se tomó la decisión de eliminar ambas columnas, también para lograr que el modelo pudiera hacer el análisis enfocándose en la forma de vida más que en el lugar que se reside.

4) Datos extremos

Finalmente se identificó que la distribución de datos en la columna de tenía algunos valores extremos.

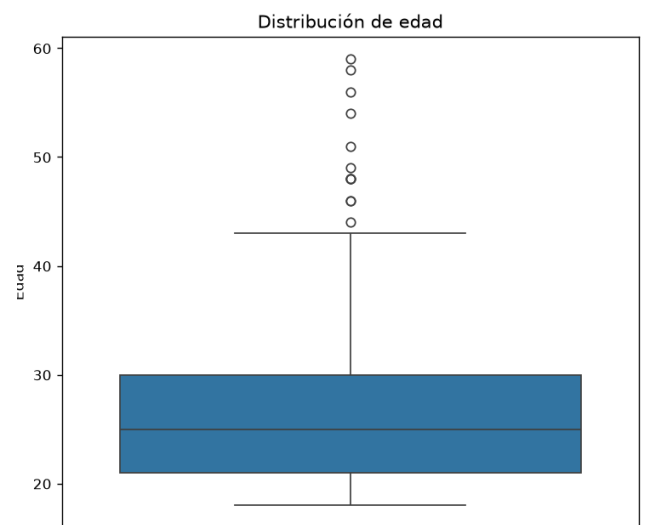


Fig. 8 Distribución de edad

Como se puede ver en la caja de bigotes la mayoría de edad esta entre los 21 años y los 30, pero hay registros de hasta 56 años. Para mantener los datos centrados y que se apegue al perfil de un estudiante se decidió mantener los datos en un rango de máximo 30 años.

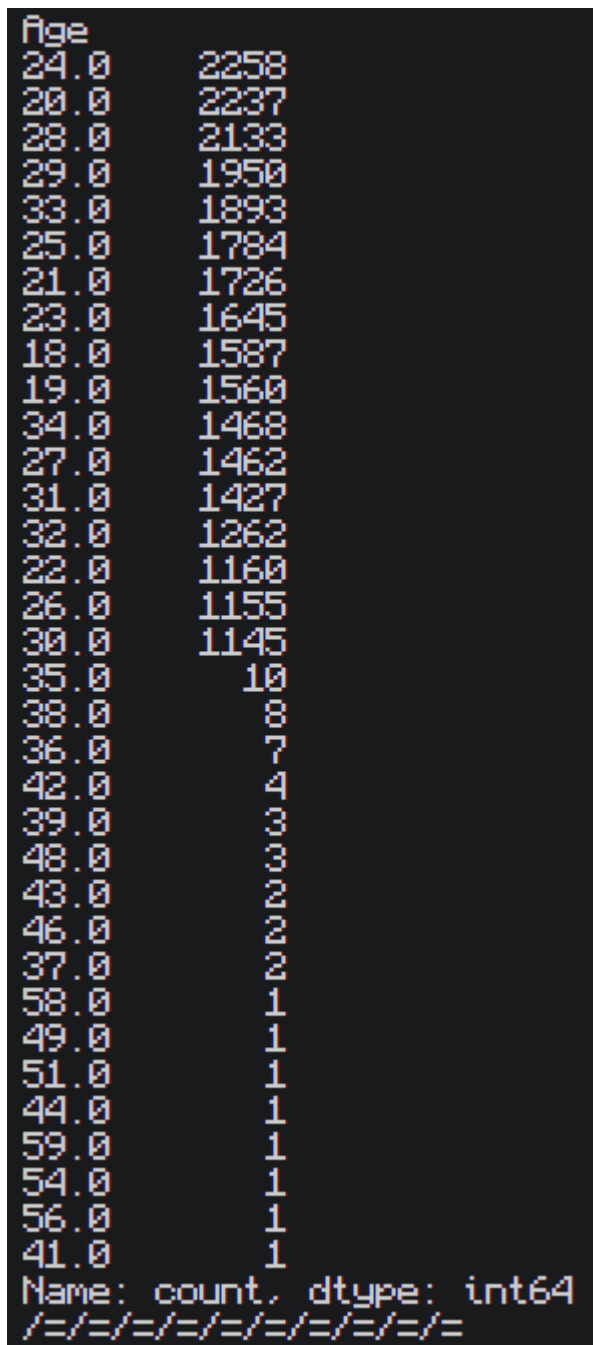


Fig. 9 Conteo de la columna "Age"

5) Transformación de los datos

Después de hacer el análisis y decidir que columnas se quedarán, se transformaron los datos para poder mantener todo en valores numéricos. Es decir, columnas como "Sleep duration" pasarían de tener valores como "Menos de 5 horas" a un valor numérico como "0". Esto para que al modelo le sea más fácil leer los datos de clasificación.

```

===== Info del Dataset =====
<class 'pandas.DataFrame'>
Index: 21802 entries, 1 to 27900
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Gender                                21802 non-null  int64
1   Age                                   21802 non-null  float64
2   Academic Pressure                    21802 non-null  float64
3   CGPA                                 21802 non-null  float64
4   Study Satisfaction                   21802 non-null  float64
5   Sleep Duration                       21790 non-null  float64
6   Dietary Habits                       21793 non-null  float64
7   Suicidal Thoughts                   21802 non-null  int64
8   Work/Study Hours                    21802 non-null  float64
9   Financial Stress                     21800 non-null  float64
10  Family Illness                       21802 non-null  int64
11  Depression                           21802 non-null  int64
dtypes: float64(8), int64(4)

```

Fig. 10 Columnas del set de datos transformados

Al hacer la transformación hay datos que se clasificaban como otros, al ser muy pocos en todos los datos se decidieron eliminarlos.

```

===== Datos faltantes =====
Null data:
Gender                                0
Age                                   0
Academic Pressure                     0
CGPA                                  0
Study Satisfaction                    0
Sleep Duration                        12
Dietary Habits                        9
Suicidal Thoughts                     0
Work/Study Hours                     0
Financial Stress                      2
Family Illness                        0
Depression                            0
dtype: int64
Null data: 23
/=/=/=/=/=/=/=/=/=/=/

```

Fig. 11 Datos otros transformados en "null data"

Finalmente, el set de datos tras ser analizado y limpiado queda con 21,779 en 11 columnas de atributos y 1 columna que funciona como el objetivo a predecir.

III. MODELO DE PREDICCIÓN

1) Presentación del modelo

Tras el extenso análisis de los datos y su limpieza es momento de elegir un modelo de predicción para resolver este enigma. Por la naturaleza de los datos a primera instancia se hace notar que el valor que intentamos predecir es 1 o 0, es decir, categórico por lo que el modelo que mejor se ajusta a estas situaciones es la regresión logística.

Este modelo consiste en aprovechar el modelo lineal y aplicar la función de la sigmoide, la cual hace que el resultado entre en un rango de 1 y 0, justo lo que necesitamos para este problema. La ecuación de la sigmoide es la siguiente.

$$f(x) = \frac{1}{1 - e^{-x}}$$

Fig. 12 Ecuación de la sigmoide

De esta manera se pueden lograr 2 cosas, primero como ya se mencionó el resultado entra en un rango de 1 a 0, el segundo es que la gráfica toma una forma distinta que se ajusta a los datos de una mejor forma.

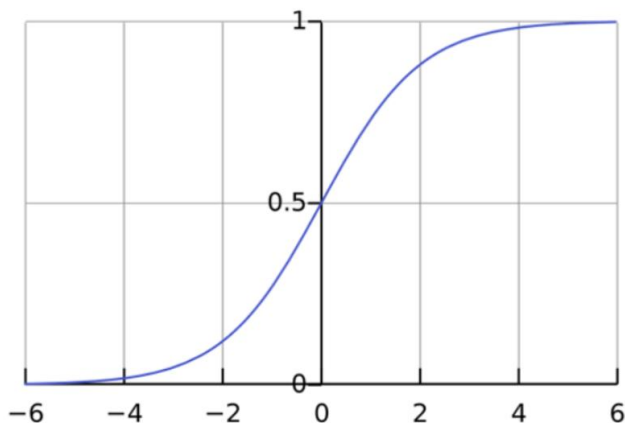


Fig. 13 Función de regresión logística

A diferencia de la regresión lineal en la que en ocasiones algunos datos eran registrados de forma incorrecta por la manera que los captaba arriba o debajo de la línea, la regresión logística permite establecer un umbral para determinar a qué categoría pertenece cada dato de manera más eficiente. De esta forma, el modelo puede separar las observaciones en las clases correspondientes y realizar predicciones más adecuadas para la naturaleza binaria de nuestro problema.

2) Optimización

Uno de los elementos más importantes del entrenamiento es el método de optimización. Al inicio del entrenamiento, nuestro algoritmo no tendrá los valores correctos para los pesos de cada variable, por lo que necesitamos un método que permita ajustarlos de manera adecuada para obtener mejores predicciones.

La gradiente descendiente es un método de optimización utilizado para minimizar la función de pérdida, ajustando los parámetros del modelo, en este caso, los pesos y el sesgo. En cada iteración, el algoritmo calcula la gradiente de la función de pérdida, la cual indica la dirección en la que se debe modificar cada parámetro para reducir el error. De esta manera, los pesos se actualizan progresivamente con el objetivo de disminuir la función de pérdida y acercarla lo más posible a cero.

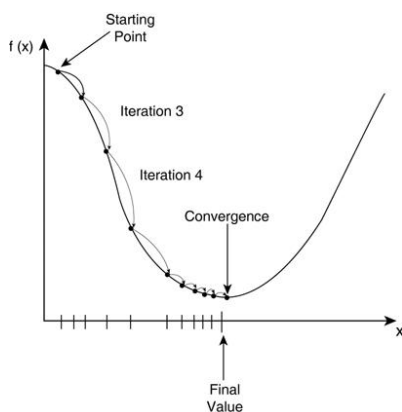


Fig. 14 Gradiente descendiente

3) Cross-Entropy

Cross-Entropy o entropía cruzada es una función que permite medir la diferencia entre dos distribuciones de probabilidad. Es decir, permite comprar que tan alejado esta la predicción de la realidad. En el caso de regresión logística, el modelo genera una probabilidad entre 0 y 1 para cada observación. La función penaliza predicciones incorrectas con un valor alto como resultado por lo que nuevamente buscamos llegar lo más cercano a 0 posible. La utilizaremos como función de pérdida para evaluar qué tan bien está prediciendo el modelo con respecto a la realidad a lo largo de las iteraciones.

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

Fig. 15 Función de cross-entropy

IV. PRIMER MODELO

1) Preparación de los datos

Para poder entrenar de forma correcta y sacar el mayor provecho a los datos que tenemos disponibles es imprescindible separarlos en distintos conjuntos que permitirán asegurar la mayor precisión de predicción por parte del modelo.

Estos conjuntos serán conocidos como “train”, “validation” y “test”. Hablemos un poco del propósito de cada uno de ellos.

- **Train:** Este conjunto de datos será el más grande y permitirá el modelo a ser entrenado y ajustar su precisión con las ecuaciones anteriormente mencionadas. Está conformado por el 80% de los datos
- **Validation:** Este conjunto nos sirve para evaluar el modelo durante el entrenamiento, nos ayudará a decidir que tan bien se ajusta el modelo al problema y detectar problemas de “overfit” o “underfit”. Es importante mencionar que estos datos no los utiliza durante su entrenamiento. Está conformado por el 10% de los datos
- **Test:** Este conjunto de datos nos permite calcular la precisión del modelo después de haber sido entrenado y con datos que no ha visto antes. Esto permite ver que tan bien funciona el modelo y que tan bien se ajustó. Está conformado por el 20% de los datos

Así mismo en todos los conjuntos se separan las columnas de la columna denominada “target” originalmente, estas serán la “X” y “Y”.

2) Descripción del modelo

Nombre	Función	Descripción
sigmoid(z)	Función de activación	Convierte el valor calculado en una probabilidad entre 0 y 1.

Cross-entropy(X, y, w, b)	Función de costo	Calcula el error del modelo mediante la entropía cruzada.
grad_fun(X, y, w, b)	Cálculo del gradiente	Calcula los gradientes necesarios para actualizar los pesos y el sesgo.
grad_des(x_train, y_train, x_val, y_val, alpha, iterations)	Función de optimización	Ajusta los pesos mediante descenso del gradiente y registra el error de entrenamiento y validación.
predict(X, w, b)	Predicción	Genera las predicciones finales del modelo, clasificando cada dato como 0 o 1.

3) Parámetros

Pesos (w): Son inicializados en 0 y actualizados durante el entrenamiento dentro de la función de optimización.

Sesgo (b): Inicializado en 0 y actualizado durante el entrenamiento dentro de la función de optimización.

4) Hiperparámetros

Learning rate (lr): Taza de aprendizaje, controla que tan grandes son los pasos de la gradiente descendiente. Esta inicializado en 0.1.

Epochs (iteraciones): Número de iteraciones para ajustar los pesos del modelo. Una sola época equivale a una vuelta completa de los datos en la función de optimización.

V. RESULTADOS DEL PRIMER MODELO

A. Resultados

El primer modelo fue entrenado en dos configuraciones diferentes, utilizando 1000 iteraciones y 500 iteraciones. Los resultados muestran que a partir de las 200 iteraciones el modelo se acercó bastante a su punto de estabilización

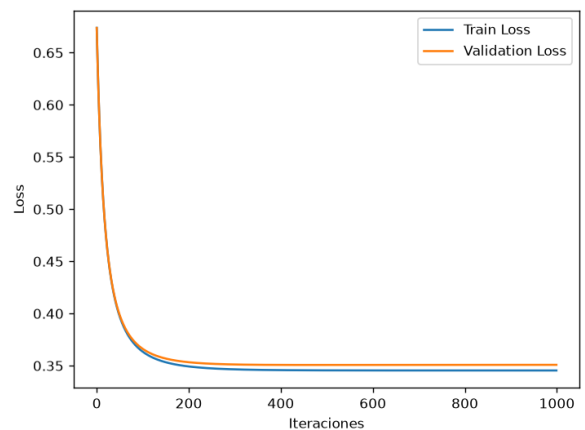


Fig. 16 Grafica de costos train contra validation en 1000 iteraciones

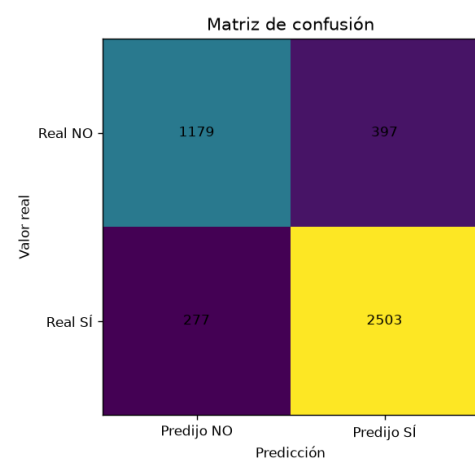


Fig. 17 Matriz de confusión del primer modelo en 1000 iteraciones

La precisión del primer modelo en 1000 iteraciones usando los datos de "test": 84.52708907254362.

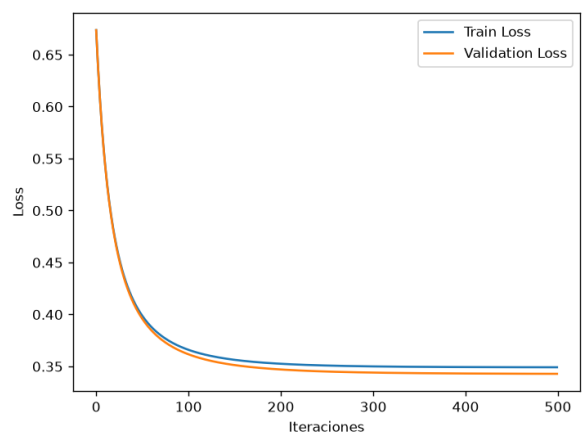


Fig. 18 Grafica de costos train contra validation en 500 iteraciones

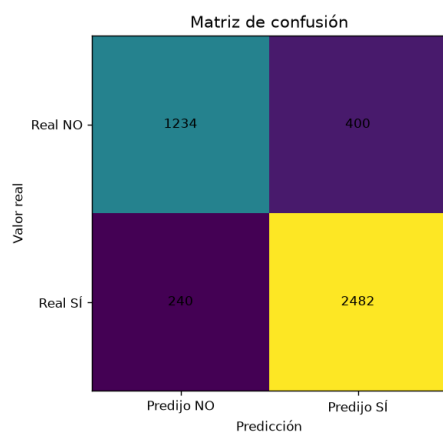


Fig. 19 Matriz de confusión del primer modelo en 500 iteraciones

La precisión del primer modelo en 500 iteraciones usando los datos de “test”: 85.30762167125803.

B. Interpretación

Como ya se mencionó el modelo alcanza una rápida disminución de la función de costo durante las primeras iteraciones para estabilizarse alrededor de las 200. Las curvas de “train” y “validation” permanecen cercanas por lo que no se observa evidencia de “overfit”. En cuanto al desempeño final en ambas versiones se alcanza más del 80% de precisión en las predicciones de “test”, la diferencia entre ambas versiones es de solamente un 0.8% aproximadamente.

En cuanto a los errores de predicción podemos ver que ambas versiones tienen una tendencia de predecir más falsos positivos (FP), en nuestro contexto significa que habría casos que personas recibirían tratamiento de depresión cuando no la tiene realmente, en cambio de los falsos negativos (FN) que son los que más nos interesa reducir pues estas personas están en riesgo de salud mental y no están recibiendo tratamiento.

C. Diagnóstico

La precisión entre los dos modelos como ya se mencionó es mayor al 80%, así mismo los errores recen más en falsos positivos esto sugiere que el modelo si se está adaptando a los datos. En nuestras graficas de costo, podemos ver que dejan de bajar considerablemente y tanto entrenamiento como validación se mantiene juntas por lo que podemos descartar una situación de “overfitting”. Sin embargo, ambos se mantienen por arriba del 0.34 lo cuál puede indicar una situación de “underfitting” pues hay cosas que el modelo no está ajustando de forma correcta o hay una falta de parámetros que permitan al modelo tener un costo menor, es importante probar con otro modelo para verificar si este valor de costo es lo más bajo o nos encontramos en una situación de “underfitting”.

VI. MODELO CON FRAMEWORK (RANDOM FOREST TREE)

Para verificar y comparar el desempeño del primer modelo de regresión, es importante compararlo con el desempeño de otro modelo. Haremos uso de un “framework” en esta ocasión, con la idea de utilizar un modelo más optimizado y estandarizado que una implementación desde 0, así sabremos si se puede predecir correctamente el problema y como ajustar el primer modelo.

1) Contexto

“Random Forest Trees” es un conjunto de métodos de ML que aprovecha la idea de los árboles de decisión y la expande creando múltiples árboles de decisión para mejorar la precisión y robustez, por eso su nombre de bosque. La idea central del bosque es que múltiples árboles entrenados de forma independiente pueden capturar mejor la relación entre los datos y reducir de esta forma la varianza en comparación a un solo árbol de decisión.

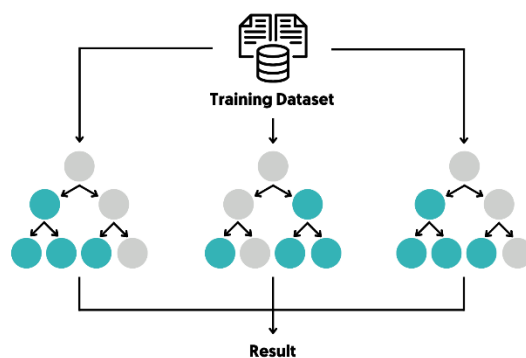


Fig. 20 Visualización de un Random Forest Tree

Este método de utilización se considera una gran mejora sobre los árboles de decisión ya que tienen a tener problemas de “overfitting” fácilmente. Al disponer de varios con distintos parámetros y varios árboles se puede promediar las predicciones y lograr un consenso de predicción más acertada.

El objetivo nuevamente es verificar el desempeño de un modelo más optimizado y robusto para solucionar el problema de análisis y determinar como optimizar el primer modelo.

2) Modelo

Para la implementación del modelo se usó la librería sklearn,

Este nuevo modelo da una precisión de 83%, un poco más baja que el primer modelo

Nos indica que no hay undefit.

Mejoras del modelo 1, lectura de datos en forma one hot encoding.

https://www.cs.us.es/~fsancho/Blog/posts/Mejorando_Gradiente_Descendiente.md

<https://www.ibm.com/mx-es/think/topics/gradient-descent>

<https://aws.amazon.com/es/what-is/machine-learning/>

<https://aws.amazon.com/es/compare/the-difference-between-machine-learning-supervised-and-unsupervised/>

<https://www.datacamp.com/es/tutorial/the-cross-entropy-loss-function-in-machine-learning>

<https://www.diegocalvo.es/funcion-de-coste-redes-neuronales/>

<https://www.ibm.com/mx-es/think/topics/random-forest>